MSIT 5910-01 MSIT Capstone Project — Final Demo and Presentation

Designing and Evaluating an Integrated Access Control Framework for Electronic Health Records in a Saudi Government Hospital

`PERMIT` `DENY` `STEP_UP_MFA` `BREAK_GLASS_PERMIT`

Taghreed Y. Almirabi

Master of Science in Information Technology, University of the People

Supervisor: Dr. Stella T. Whyte

October 2026

Introduction and problem statement

Context

Saudi government hospitals are moving to shared electronic health records, where physicians, nurses, pharmacists, laboratory staff and administrators all touch the same patient data.

At the same time the Personal Data Protection Law and the National Cybersecurity Authority's Essential Cybersecurity Controls raise the bar for who may see what, when, and with what evidence.

Clinicians still need a record in seconds, and an emergency cannot wait for a permission request.

The problem

Role permissions, authentication, clinical context, emergency access and audit review are enforced by separate tools, so access is either too broad or blocked when minutes matter, and the evidence is incomplete.

Purpose of the project

Design and evaluate one integrated framework that decides every request by role, context and risk, grants controlled emergency access, and keeps a tamper-evident audit trail.

Aim, objectives and scope

Aim: design, implement and evaluate one integrated access-control framework that coordinates identity assurance, role permissions, clinical context, emergency access and tamper-evident audit evidence in a single explainable decision flow, following the design science research methodology.

01 Map six hospital roles to the minimum necessary EHR actions.

02 Define authentication, authorization, context, emergency and audit requirements with testable acceptance measures.

03 Implement a rules-based Python prototype that evaluates synthetic requests and persists signed audit evidence.

04 Test normal, denied, high-risk, emergency and misuse scenarios at unit, integration, system and acceptance levels, and measure local performance.

05 Obtain structured feedback on clarity and practicality: a developer heuristic plus validation by two healthcare IT experts.

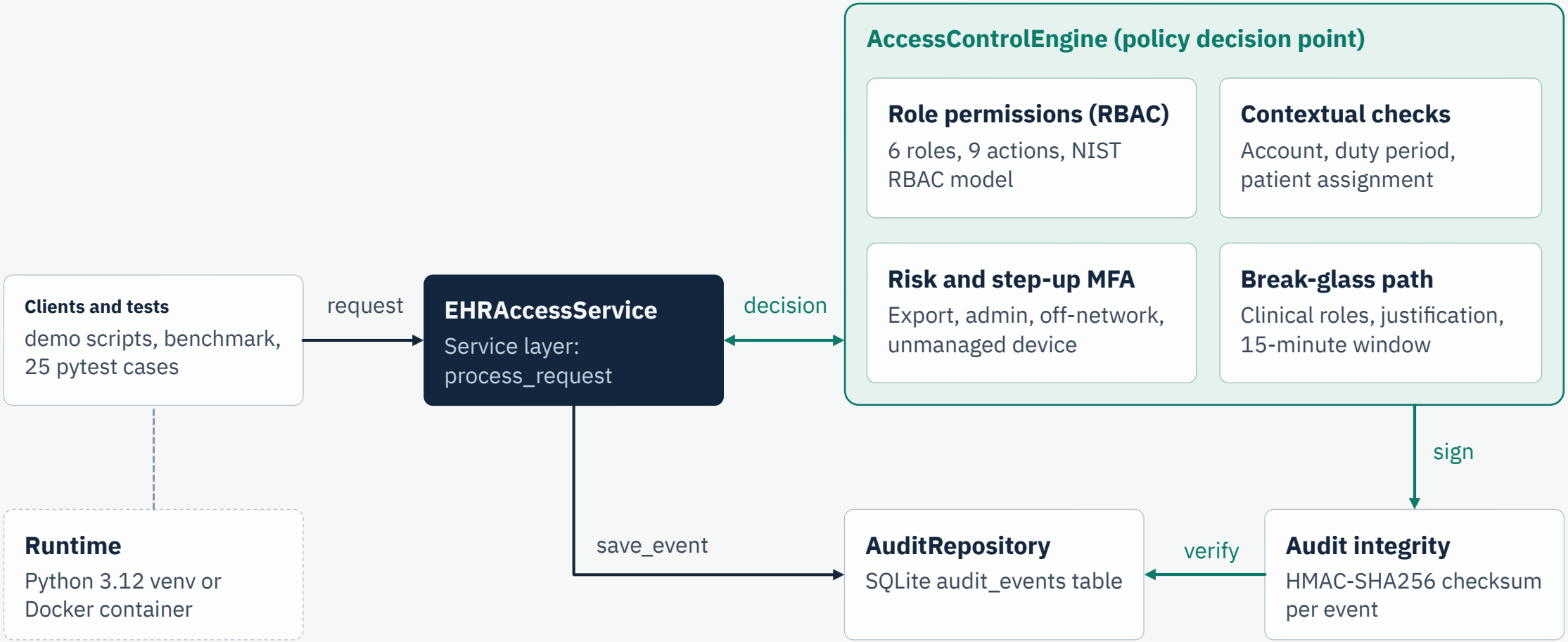
In scope

- Policy engine, audit repository and service layer
- Six clinical and technical roles, nine actions
- Automated tests, 36-case benchmark, expert validation
- Deployment plan (Docker) and maintenance plan

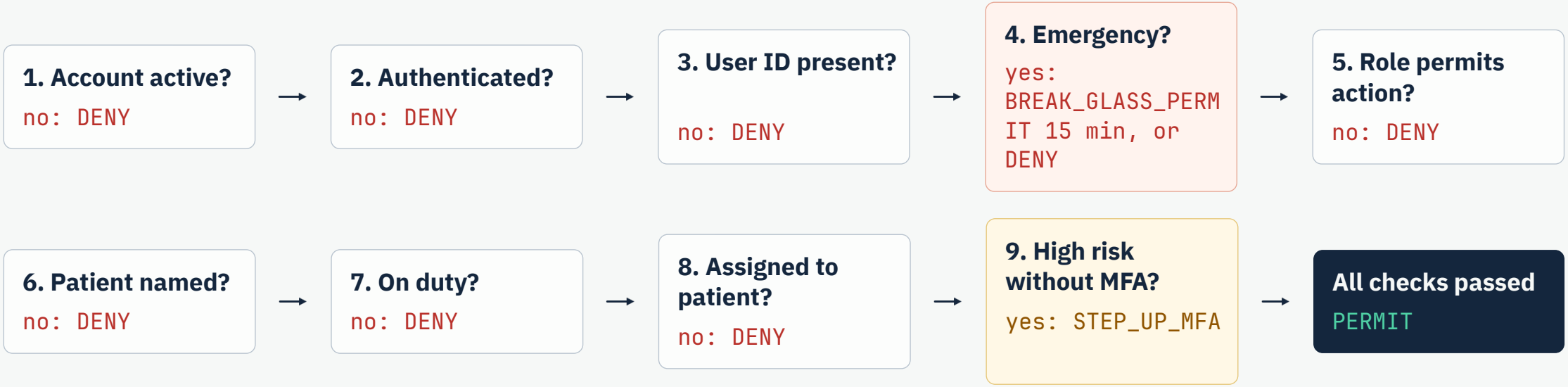
Out of scope

- Real patient data or a live hospital system
- Integration with a production identity provider
- A graphical user interface beyond the command line

System architecture: one policy engine, one audit trail



How a request is decided: nine checks, first failure stops



High risk means an export or account-management action, a request from outside the hospital network, or an unmanaged device. Break-glass is limited to physicians and nurses, requires a named patient, a clinical action and a written justification of at least ten characters, and is flagged for review. Every outcome, including every denial, is written to the audit log with its reason.

Components: role matrix and the four decisions

Role	Permitted actions	Needs MFA
Physician	View record, update clinical notes, prescribe medication	Context only
Nurse	View record, update clinical notes, administer medication	Context only
Pharmacist	View record, dispense medication	Context only
Laboratory technician	View record, record laboratory result	Context only
Health information manager	View record, export record, review audit logs	Export
IT systems administrator	Manage user accounts, review audit logs (no clinical data)	Accounts

“Context only” means MFA is required when the request comes from outside the hospital network or from an unmanaged device.

PERMIT
Role and contextual checks passed.

DENY
Stopped at a failed check; the reason is recorded.

STEP_UP_MFA
Allowed only after a second authentication factor.

BREAK_GLASS_PERMIT
Temporary emergency access, expires in 15 minutes, flagged for review.

Technologies, methods and standards

Implementation

- Python 3.12 with dataclasses and enums for typed requests and decisions
- SQLite audit repository through the standard library
- HMAC-SHA256 checksums with hmac and hashlib
- pytest 9.1.1 for 25 automated tests
- perf_counter and tracemalloc for latency and memory
- Matplotlib for the evaluation chart

Methodology and workflow

- Design science research methodology: problem, objectives, design, demonstration, evaluation, communication
- Iterative unit-by-unit development with Git: main and development branches, pull-request review
- Six semantic-version tags, v0.1.0-core to v1.3.0-final
- Unit, integration, system, acceptance and stability test levels
- Docker image, deployment guide and environment template; CI/CD workflow planned for every pull request

Standards followed

- NIST RBAC model for roles and permissions
- NIST SP 800-63B-4 for authentication assurance and step-up MFA
- NIST SP 800-92 for audit log management
- ISO/IEC 25010:2023 product quality model
- ISO/IEC/IEEE 29119 software testing
- Saudi PDPL and NCA ECC-2:2024 requirements

Implementation: the decision code and what it produces

src/access_control_engine.py (excerpt)

```
def evaluate(self, request):
  if not request.account_active:
    return self._finish(request, DENY,
      "Account is inactive.")
  if not request.authenticated:
    return self._finish(request, DENY,
      "Authentication is required.")
  if request.emergency:
    return self._evaluate_break_glass(request)
  allowed = self.PERMISSIONS.get(request.role)
  if request.action not in allowed:
    return self._finish(request, DENY, ...)
  ...
  elevated_risk = (action in HIGH_RISK_ACTIONS
    or not request.on_hospital_network
    or not request.managed_device)
  if elevated_risk and not request.mfa_verified:
    return self._finish(request, STEP_UP_MFA, ...)
  return self._finish(request, PERMIT, ...)
```

Output of demo.py (synthetic data)

```
SCENARIO: Unassigned nurse views a record
ROLE: nurse  ACTION: view_patient_record
DECISION: DENY
REASON: The clinician is not assigned
  to this patient.
AUDIT EVENT: c3b6427c35b750ba

SCENARIO: Physician activates break-glass
DECISION: BREAK_GLASS_PERMIT
REASON: Temporary emergency access
  granted and flagged for review.
EMERGENCY ACCESS EXPIRES: +15 minutes

AUDIT INTEGRITY CHECK
ORIGINAL EVENT VALID: True
ALTERED EVENT VALID:  False
TOTAL AUDIT EVENTS: 5
```

Testing: four levels, twenty-five automated cases

Level	Tests	What it verifies
Unit	15	Each rule in isolation: permit, deny, MFA, break-glass, audit checksum detects tampering
Integration	3	Engine, service layer and SQLite repository together: every decision is persisted
System and acceptance	6	End-to-end healthcare workflows: blank user denied, physician permitted, export needs MFA, break-glass time-limited, IT admin denied clinical access
Stability	1	Fifty requests processed with no lost audit events

White-box tests cover the policy branches; black-box scenario tests exercise the complete prototype through its inputs and outputs. Windows 11, Python 3.12.15, pytest 9.1.1.

25 / 25

tests passed on v1.2.0-unit7 (0.44 s) and again on v1.3.0-final (0.41 s)

Evidence kept in the repository

- Full pytest log for the release
- Test files at each level
- Tag and commit hash tie the log to the code

Results: every target met with wide margins

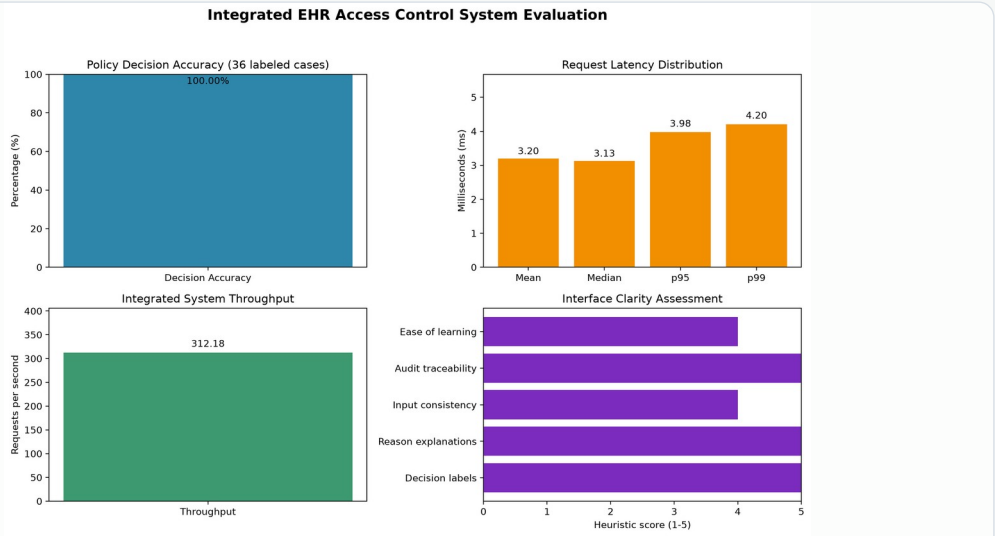
100%
decision accuracy on 36 labelled scenarios, 0 false permits

3.196 ms
mean latency per audited decision (p99 4.205 ms)

312
requests per second, each written to SQLite

0.127 MB
peak memory during the run

4.85 / 5
expert validation by two healthcare IT reviewers



How the numbers were produced

- Accuracy: 36 labelled cases, six per role, compared with expected decisions fixed in advance; false permits counted (0)
- Latency and throughput: 10 warm-up requests, then 300 requests each timed with perf_counter and persisted to SQLite; mean 3.196 ms, median 3.131, p95 3.976, p99 4.205
- Memory: tracemalloc peak during the timed run; environment recorded (Windows 11, Python 3.12.15, 12 logical CPUs, 7.7 GB)
- Clarity: heuristic review 4.60 / 5 (chart, lower right); expert review 4.85 / 5 on security behaviour, clarity and applicability, both reviewers recommending a pilot
- Target: NFR-01 response time under 250 ms, exceeded by a factor of about 60

Source: results/evaluation_results.csv and performance_evaluation.py at release v1.3.0-final (commit 0bb3017)

Objectives against evidence

Objective	Evidence produced	Status
O1	Six-role least-privilege matrix implemented and tested; every prohibited role–action pair denied (100%)	Met
O2	Nine requirements with acceptance measures, each traced to at least one automated test (100% traceability), anchored to NIST RBAC, SP 800-63B-4 and SP 800-92	Met
O3	Working prototype with signed, persisted audit evidence; releases v1.2.0-unit7 and v1.3.0-final (commit 0bb3017), synthetic data only, 25 passing tests	Met
O4	100% accuracy on 36 labelled cases with zero false permits, 3.196 ms mean and 4.205 ms p99 latency, 312 requests per second, 0.127 MB peak memory, results read against the ISO/IEC 25010 quality model	Met, all targets exceeded
O5	Heuristic clarity review 4.60 / 5; two expert healthcare IT reviewers rated the prototype 4.85 / 5 (target 4.0) and both recommended a pilot	Met

Each objective is answered by an artifact in the repository or the report, so the claim can be checked rather than trusted.

Delivery: six tagged releases in eighteen days

`v0.1.0-core`
7 Oct 2026. Core access-control engine

`v0.2.0-tested`
8 Oct 2026. Unit tests for every rule

`v1.0.0`
8 Oct 2026. Documented release with README

`v1.1.0-unit6`
8 Oct 2026. Integration evaluation, benchmark, Docker plan

`v1.2.0-unit7`
17 Oct 2026. System testing, stability run, maintenance plan

`v1.3.0-final`
24 Oct 2026. 36-case benchmark, latency percentiles, environment record

Version control

Feature work on development, merged to main through a reviewed pull request; each tag carries its full test log and commit hash.

Deployment

Dockerfile and deployment guide for an on-premise server or private cloud; environment template for paths, log level and audit secret.

Maintenance

Testing and maintenance plan: corrective, adaptive, perfective and preventive tasks, with a CI/CD workflow planned to run the full suite on every pull request.

Conclusion and future enhancements

What the project showed

Role, context and risk can be combined in one small, readable policy engine without slowing the clinician: a decision costs under four milliseconds including its audit write.

Emergency access and accountability are not in conflict when break-glass is time-limited, justified and flagged, and every event carries a checksum that exposes tampering.

Measurable objectives, tagged releases and committed test evidence made the work verifiable, which mattered more than any single feature.

Next steps

- Add an inactivity timeout and an explicit close action for break-glass sessions, so an abandoned emergency session ends before the 15-minute limit (NIST SP 800-63B-4)
- Load and concurrency testing, independent trials with confidence intervals, and validation with clinicians in approved workflow sessions
- Connect to a hospital identity provider such as Nafath and a real MFA authenticator at the assurance level defined in SP 800-63B-4
- Break-glass review dashboard, cached context for degraded networks, and audit export to standard SIEM connectors, as the reviewers recommended
- Expose the engine as an authenticated API behind the EHR, read the documented environment variables, and pilot with a hospital IT security team before any production consideration

References

Almaghrabi, N. S., & Bugis, B. A. (2022). Patient confidentiality of electronic health records: A recent review of the Saudi literature. *Dr. Sulaiman Al Habib Medical Journal, 4*(3), 126–135. https://doi.org/10.1007/s44229-022-00016-9

Cobrado, U. N., Sharief, S., Regahal, N. G., Zepka, E., Mamauag, M., & Velasco, L. C. (2024). Access control solutions in electronic health record systems: A systematic review. *Informatics in Medicine Unlocked, 49*, Article 101552. https://doi.org/10.1016/j.imu.2024.101552

Ferraiolo, D. F., Sandhu, R., Gavrila, S., Kuhn, D. R., & Chandramouli, R. (2001). Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security, 4*(3), 224–274. https://doi.org/10.1145/501978.501980

International Organization for Standardization. (2023). Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Product quality model (ISO/IEC Standard No. 25010:2023). https://www.iso.org/standard/78176.html

Kent, K., & Souppaya, M. (2006). Guide to computer security log management (NIST Special Publication 800-92). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-92

National Cybersecurity Authority. (2024). Essential cybersecurity controls (ECC-2:2024). https://nca.gov.sa/en/regulatory-documents/controls-list/ecc/

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Saudi Data and Artificial Intelligence Authority. (2023). Guide to the Saudi Personal Data Protection Law for controllers and processors. https://dgp.sdaia.gov.sa/wps/portal/pdp/knowledgecenter/details/PDPLCP

Temoshok, D., Fenton, J. L., Choong, Y.-Y., Lefkovitz, N., Regenscheid, A., Galluzzo, R., & Richer, J. P. (2025). Digital identity guidelines: Authentication and authenticator management (NIST Special Publication 800-63B-4). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-63B-4